Dans l'interface Frigate, j'ai cliqué sur la caméra NetCam1 pour l'afficher en plein écran. En bas de l'écran, les boutons PTZ sont apparus : ← → ↑ ↓ pour le mouvement, et les boutons + et - pour le zoom. J'ai utilisé ces contrôles pour orienter la caméra vers la plaque RT-1994-LP jusqu'à ce qu'elle soit bien visible au centre de l'image.

2. Pour activer la reconnaissance de plaques, j'ai configuré la section LPR (License Plate Recognition) globalement et sur la NetCam1 :

```
lpr:
  enabled: true
  device: CPU
  detection_threshold: 0.7
  recognition_threshold: 0.8
  min_area: 500
```

- `enabled: true` → active la fonctionnalité LPR.
- `device: CPU` → utilise le processeur pour l'analyse.
- `detection_threshold: 0.7` → le seuil signifie que Frigate doit être sûr à 70 % qu'il y a une plaque.
- `recognition_threshold: 0.8` → ce seuil demande 80 % de confiance pour lire les caractères.
- `min_area: 500` → définit la taille minimale de la plaque en pixels.

Ensuite, j'ai configuré NetCam1 comme caméra dédiée LPR :

```
cameras:
  NetCam1:
    type: lpr
    enabled: true
    lpr:
      enabled: true
      enhancement: 3
    objects:
      track: []
...
```

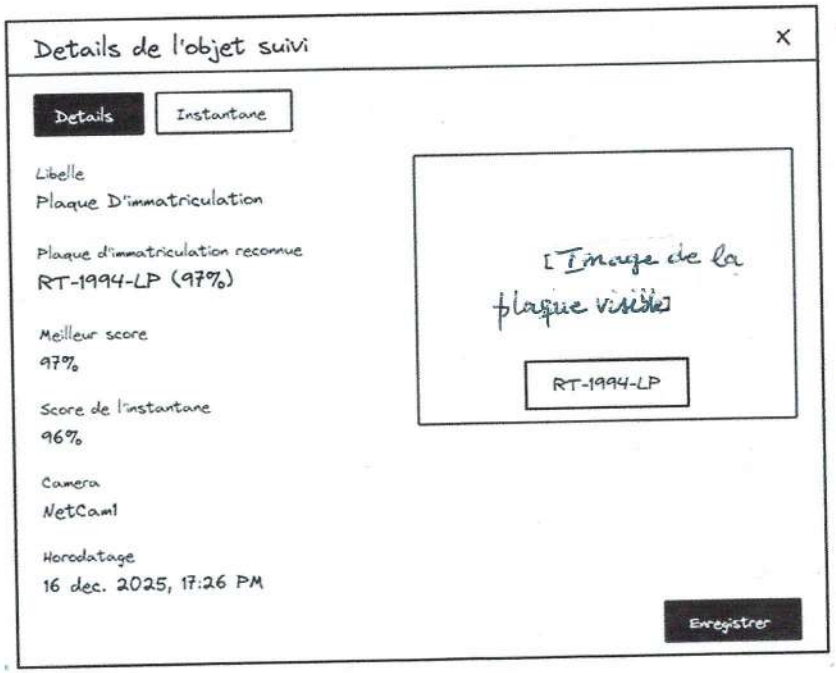
- `type: lpr` → indique que cette caméra est spécialisée pour la lecture de plaques.
- `enabled: true` → active la reconnaissance sur cette caméra.
- `enhancement: 3` → ce paramètre 'enhancement: 3' améliore la netteté de l'image pour mieux lire les caractères
- `track: []` → la liste est vide car je ne veux pas détecter d'autres objets sur cette caméra.

Après avoir redémarré Frigate, la caméra 1 affiche la plaque de test, j'observe



3. Pour voir les détails d'une plaque reconnue, je clique sur "explorer" : 🔍, la loupe à gauche, je clique sur la détection "plaque d'immatriculation", sur une image, une fenêtre popup s'ouvre avec toutes les informations. Croquis de ce que j'ai observé :

VUE DETAILS DE L'OBJET SUIVI (PLAQUE)



Quand je clique sur une détection de plaque, la fenêtre "Details de l'objet suivi" s'affiche. Je vois le libellé "plaque d'immatriculation", la plaque reconnue avec son score, le meilleur score obtenu, la caméra source et l'horodatage. L'image à droite montre la voiture avec la plaque. Cette vue permet de vérifier que la reconnaissance est correcte.

4. Pour démontrer le fonctionnement avec plusieurs plaques différentes, j'ai remplacé le flux de la caméra par un fichier vidéo contenant des véhicules avec diverses plaques.
D'abord, j'ai configuré le flux vidéo dans go2rtc :

```
go2rtc:
  streams:
    netcam1_video:
      - ffmpeg:/config/videos/plaques_video.mp4
```

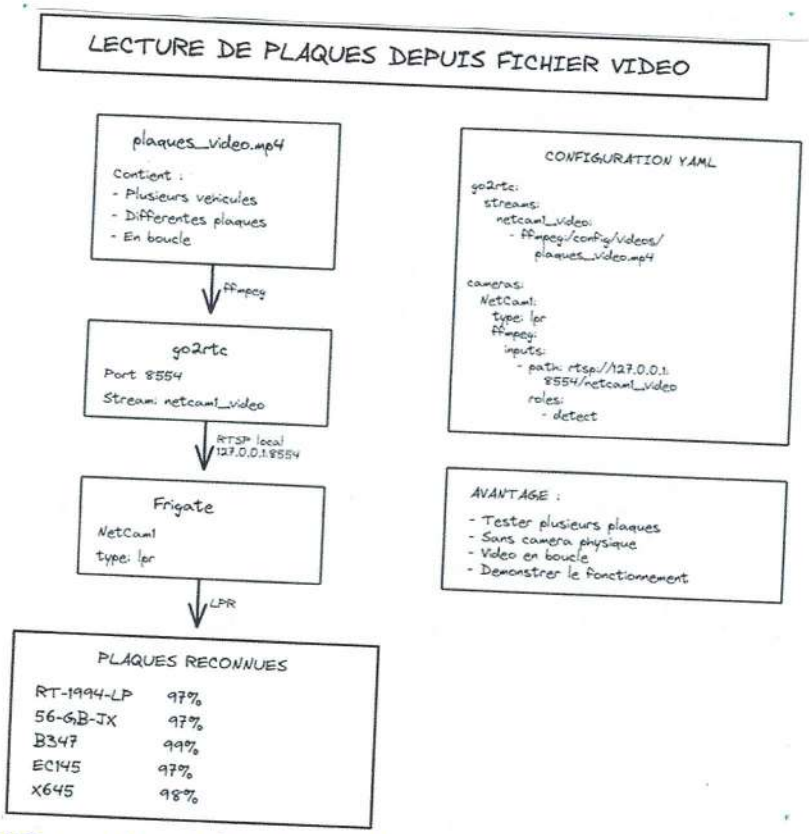
le chemin FFmpeg:....mp4. indique que go2rtc doit lire un fichier vidéo au lieu du flux RTSP. le fichier plaques_video.mp4 est placé dans le dossier /config/videos/ du conteneur Frigate. FFMPEG convertit la vidéo en flux RTSP accessible localement.

Ensuite, j'ai modifié l'entrée de NetCam1 pour utiliser ce flux :

le chemin `rtsp://127.0.0.1:8554/netcam1_video` pointe vers le flux créé par go2rtc à partir du fichier vidéo. le port 8554 est le port RTSP de go2rtc. Le nom "netcam1_video" correspond au stream défini dans go2rtc.

Dans le fichier de config, ça donne :

```
cameras:
  NetCam1:
    type: lpr
    enabled: true
    ffmpeg:
      inputs:
        - path: rtsp://127.0.0.1:8554/netcam1_video
          roles:
            - detect
  lpr:
    enabled: true
    enhancement: 3
```



La vidéo tourne en boucle et Frigate détecte chaque plaque qui passe. Dans la vue "Explorer", je vois la liste de toutes les plaques reconnues avec leur score de confiance. Les scores sont très élevés, entre 97% et 99%, ce qui montre que la reconnaissance fonctionne très bien. Je peux cliquer sur chaque détection pour voir les détails et vérifier que les caractères sont corrects.

Ce que j'observe dans l'interface réelle :



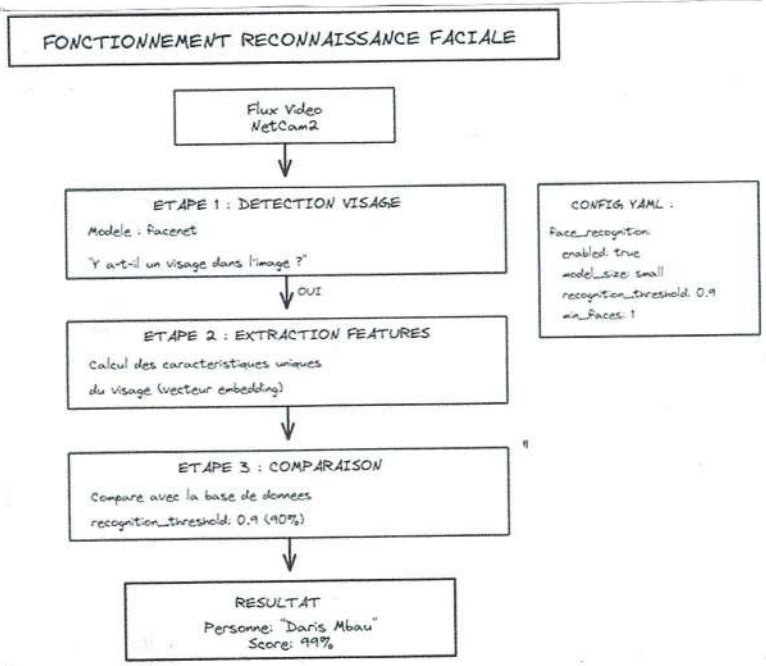
5.3. Reconnaissance faciale

1. Pour activer la reconnaissance faciale dans Frigate, j'ai configuré la section face_recognition dans le fichier config.yml:

```
face_recognition:
  enabled: true
  model_size: small
  recognition_threshold: 0.9
  min_faces: 1
...
```

→ Ce paramètre active la fonctionnalité de reconnaissance faciale.
→ ce paramètre utilise un modèle leger qui consomme moins de CPU.
→ demande 90% de confiance pour identifier une personne.
Ce paramètre signifie qu'une seule photo suffit pour commencer à reconnaître quelqu'un.

La reconnaissance faciale fonctionne en deux étapes. D'abord, Frigate détecte les visages dans l'image. Ensuite, il compare chaque visage détecté avec les visages enregistrés dans sa base de données.

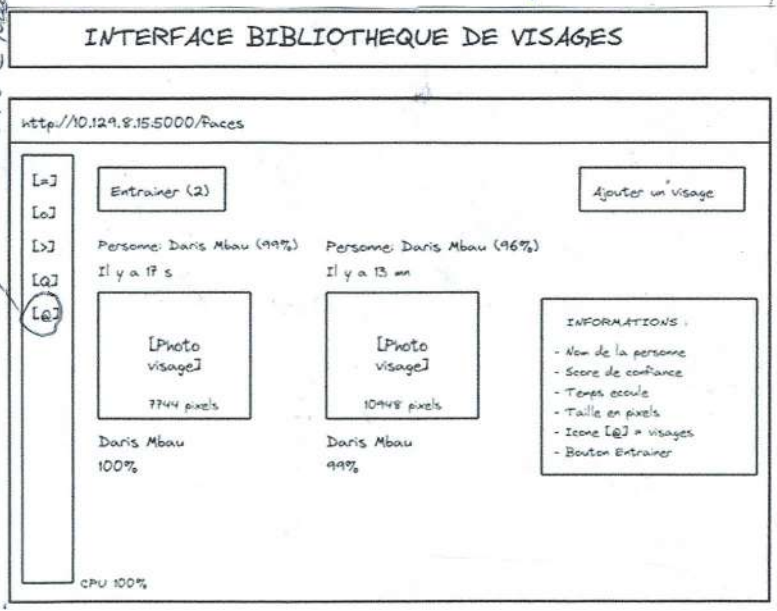


J'observe qu'après avoir activé la reconnaissance faciale, Frigate télécharge automatiquement le modèle facenet. Je le vois dans les logs, le message "facenet-facenet.onnx: downloaded". La fonctionnalité est maintenant prête à être utilisée.

2.

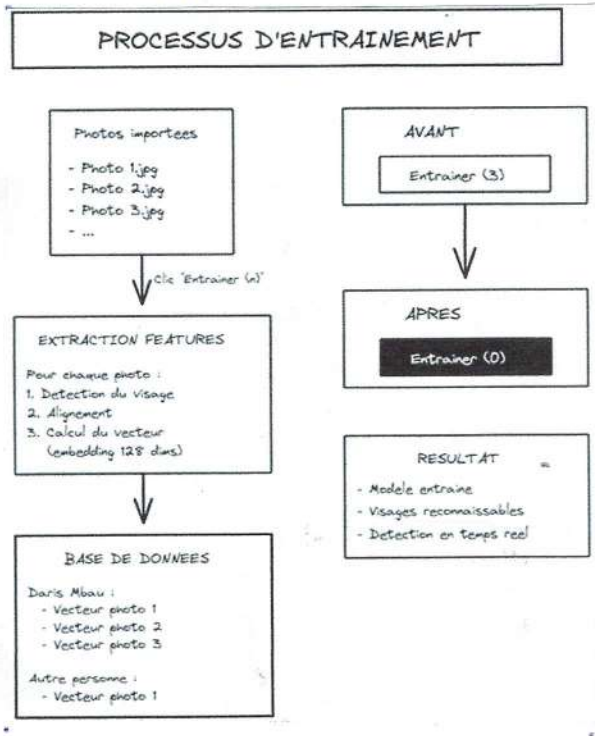
Pour que Frigate puisse reconnaître une personne, je dois d'abord lui fournir des photos de référence. J'accède à la bibliotheque de visages dans l'interface web, dans le menu de la reco. faciale à gauche. L'URL de la page devient : .../faces.

Pour ajouter un nouveau visage, je clique sur le bouton "Ajouter un visage" en haut à droite de l'interface, je peux ensuite importer plusieurs photos de la même personne. Et bien sûr, les images doivent être claires et de face.



J'observe dans la page "Bibliothèque de visages.", je vois les photos que j'ai ajoutées. Chaque vignette montre le visage détecté, le nom de la personne et le score de confiance. Le bouton "Entraîner" en haut à gauche affiche de photos en attente d'entraînement.

3. Après avoir ajouté des photos de visages. je dois entraîner le modèle pour qu'il apprenne à reconnaître ces personnes. L'entraînement permet d'extraire les caractéristiques uniques de chaque visage.
Pour lancer l'entraînement, je clique sur le bouton "Entraîner" qui affiche le nombre de photos en attente. Par exemple. "Entraîner (2)" signifie que 2 photos n'ont pas encore été traitées.
Le processus d'entraînement est rapide car le modèle "small" est léger. Frigate calcule un vecteur de caractéristiques pour chaque visage. Ces vecteurs sont ensuite utilisés pour comparer avec les visages détectés en temps réel.

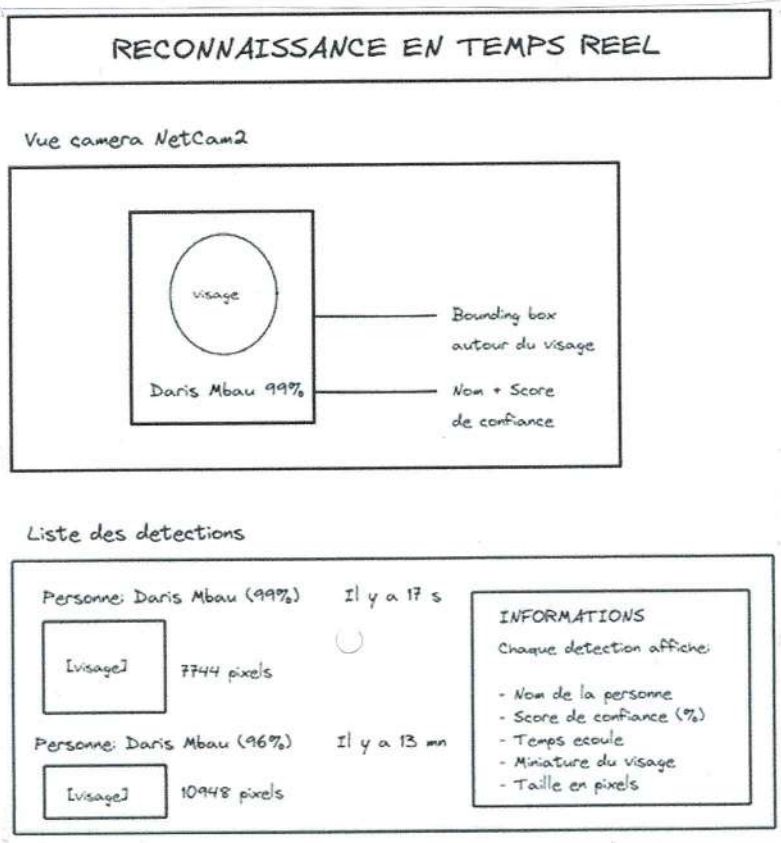


J'observe que quand je clique sur "Entraîner", le nombre entre parenthèses diminue progressivement. J'entraîne en spécifiant la personne concernée, enregistrée en amont.
Une fois l'entraînement terminé, le compteur affiche (0).

4. Une fois le modèle entraîné, Frigate reconnaît automatiquement les personnes dans le flux vidéo. Quand une personne connue passe devant la caméra, son nom s'affiche avec le score de confiance (Onglet Débogage dans les paramètres).

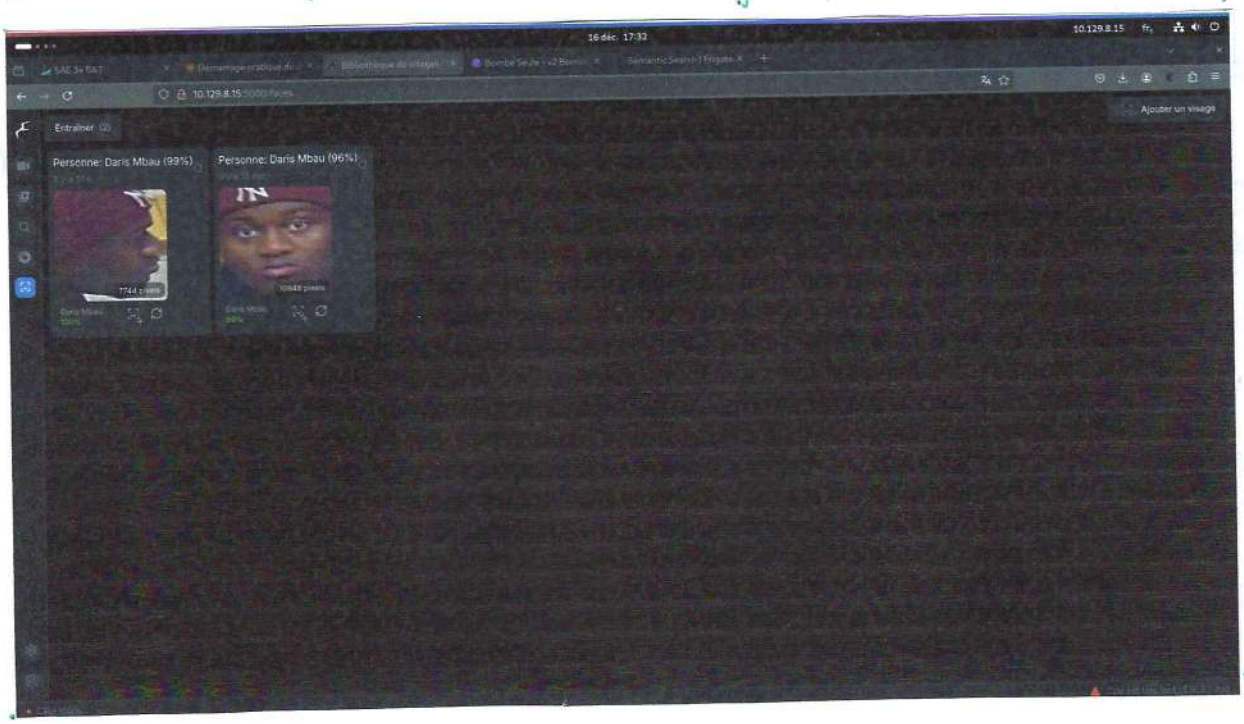
La reconnaissance s'affiche dans plusieurs endroits de l'interface :
→ Dans la vue temps réel, dans l'onglet "Débogage", avec un rectangle autour du visage.

- Dans la liste des détections avec le nom de la personne
- Dans la page "Explorer" (menu gauche) pour filtre par personne.



J'observe quand je passe devant la caméra NetCam 2, Frigate me reconnaît et affiche "Daris Mbau" avec un score de 99%. Dans la page "Bibliothèque de visages", je vois l'historique de toutes les reconnaissances.

Chaque entrée montre la photo capturée, le nom identifié, le score de confiance et le temps écoulé. La taille en pixels indique la qualité de la capture. Réellement, j'ai :



5. Pour valider le fonctionnement de la reconnaissance faciale avec l'ensemble du groupe, nous avons partagé nos photos directement via l'interface Frigate. Le processus est simple et ne nécessite pas de manipulation de fichiers.

→ Ajout des photos dans Frigate :

J'ai d'abord pris des photos de mes collègues avec mon téléphone. J'ai ensuite transféré ces photos sur l'ordinateur. Dans l'interface Frigate, j'ai accédé à la page de reconnaissance faciale. J'ai cliqué sur le bouton "Ajouter un visage" en haut à droite. Une fenêtre s'est ouverte pour importer les photos. J'ai sélectionné les photos de chaque collègue et je les ai associées à un nom.

→ Entraînement automatique :

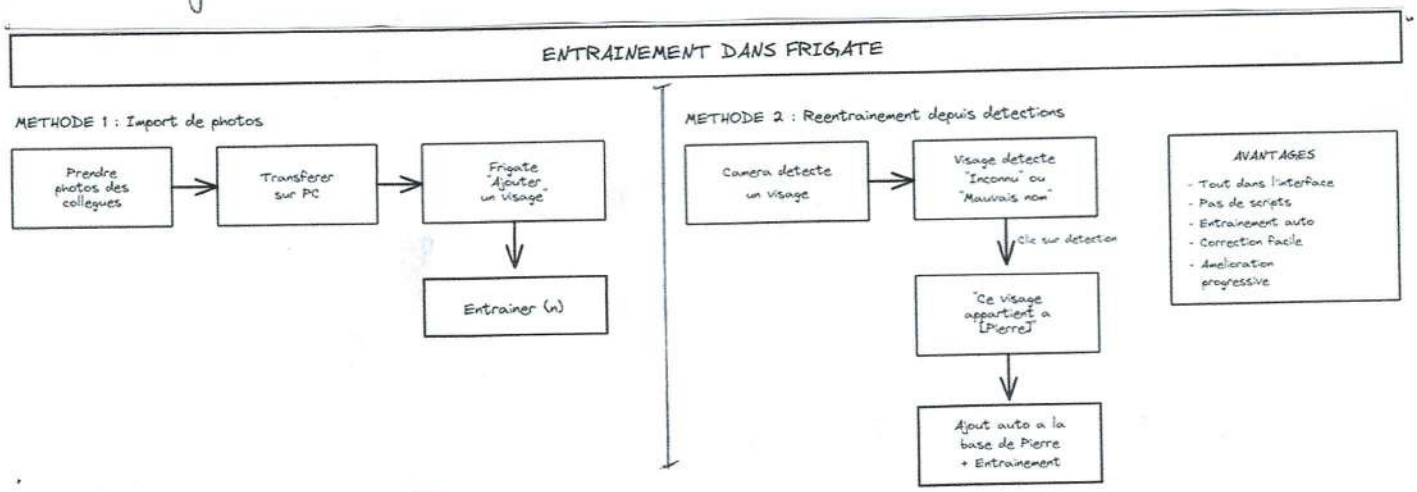
Après l'ajout des photos, Frigate propose automatiquement d'entraîner le modèle. Le bouton "Entraîner(n)" affiche le nombre de photos en attente. J'ai cliqué sur ce bouton et l'entrainement s'est lancé directement.

→ Réentraînement à partir des détections :

Une fonctionnalité très pratique de Frigate est la possibilité de réentraîner le modèle à partir des visages déjà détectés par les caméras. Quand Frigate détecte un visage inconnu, je peux cliquer dessus et lui attribuer un nom et le système ajoute cette image à la base d'entraînement. Cela améliore progressivement la reconnaissance.

→ Correction des erreurs :

Si Frigate se trompe et attribue un mauvais nom à un visage, je peux corriger l'erreur de la même manière. Je clique sur la détection incorrecte, je sélectionne le bon nom et je relance l'entraînement.



J'observe, après avoir ajouter les photos du groupe, après quelques détections, le système reconnaît correctement chaque personne du groupe. Quand un collègue passe devant la caméra, son nom s'affiche avec le score. Les erreurs de reconnaissance sont rares et faciles à corriger grâce au réentraînement.

Fonctionnement de l'algorithme de reconnaissance faciale.

Pour mieux comprendre comment Frigate reconnaît les visages, j'ai étudié l'algorithme utilisé. Frigate utilise le modèle FaceNet pour la reconnaissance faciale. Voici les étapes du processus :

1. Détection du visage : Frigate utilise d'abord un détecteur de visages pour localiser les visages dans l'image. Ce détecteur trouve les coordonnées du rectangle contenant chaque visage. Cette étape est rapide et permet de cibler uniquement les zones intéressantes de l'image.

2. Alignement du visage : Une fois le visage détecté, l'algorithme l'aligne. L'alignement consiste à redresser le visage pour qu'il soit bien droit et centré. Cette normalisation est importante. Car elle permet de comparer des visages pris sous différentes angles.

3. Extraction des caractéristiques (Embedding) : Le modèle FaceNet transforme l'image du visage aligné en un vecteur de 128 nombres. Ce vecteur s'appelle "embedding". Il représente les caractéristiques uniques du visage : forme des yeux, distance entre les yeux, forme du nez, contour du visage, etc. Deux photos de la même personne produiront des embeddings très similaires.

4. Comparaison avec la base de données : Quand Frigate détecte un nouveau visage, il calcule son embedding puis le compare avec tous les embeddings stockés dans sa base de données. La comparaison utilise la distance euclidienne ; plus les vecteurs sont proches, plus les visages sont similaires. Si la distance est inférieure au seuil recognition_threshold (0.9 dans ma configuration), Frigate identifie la personne :

- Formule de similarité :

La distance entre deux embeddings A et B se calcule ainsi :

$$\text{distance} = \sqrt{[(a1-b1)^2+(a2-b2)^2+\ldots+(a128-b128)^2]}$$

Si distance < seuil, alors les deux visages appartiennent à la même personne.

ALGORITHME DE RECONNAISSANCE FACIALE (FaceNet)

Image camera
[visage dedans]

↓ ETAPE 1: Detection

Detecteur de visages
Localise le rectangle
(x1, y1, x2, y2)

↓ ETAPE 2: Alignement

Alignement du visage

/ → | droit

↓ ETAPE 3: Embedding

FaceNet extrait
les caracteristiques

Visage -> Vecteur 128D
[0.12, -0.34, ..., 0.78]

↓ ETAPE 4: Comparaison

Comparaison avec BDD

Embedding detecte: [0.12, -0.34, ...]

Base de donnees:
  Daris: [0.11, -0.35, ...] dist=0.05
  Pierre: [0.45, 0.12, ...] dist=0.82

Seuil: 0.9 | Plus proche: Daris
RESULTAT: "Daris Mbau" (99%)

FORMULE DISTANCE

distance = racine(
  (a1-b1)^2 + (a2-b2)^2
  + ... + (a128-b128)^2
)

Si distance < seuil
-> Meme personne

QU'EST-CE QU'UN EMBEDDING?

Vecteur de 128 nombres
qui represente les
caracteristiques uniques:

- Forme des yeux
- Distance entre yeux
- Forme du nez
- Contour du visage

CONFIGURATION

face_recognition:
  model_size: small (facenet)
  recognition_threshold: 0.9
  -> Confiance minimum 90%

Ce que j'observe : Le score de confiance affiché dans l'interface représente la similarité entre le visage détecté et le visage le plus proche dans la base. Un score élevé signifie que les embeddings sont très proches. Le paramètre "recognition_threshold: 0.9" dans ma configuration signifie que Frigate n'identifie une personne que s'il est sûr à au moins 90%.

5.4. Recherche sémantique

La recherche sémantique permet de chercher des événements en utilisant des mots simples. Au lieu de filtrer par type d'objet ou par date, je peux décrire ce que je cherche. Par exemple "personne avec un vélo" et Frigate trouve les images qui correspondent.

→ J'ouvre le fichier de configuration de Frigate : nano /home/total/frigate/Configurat°/config.yml

→ J'ajoute la section suivante dans le fichier config.yml :

```
semantic_search:
  enabled: true
  model: jinav1
  model_size: small
```

→ Ce paramètre active la recherche sémantique
→ indique le model utilisé pour comprendre le sens de mots et des images.
→ Ce paramètre utilise la version légère du modèle. Ça consomme moins de ressources CPU.

→ Je sauvegarde le fichier (Ctrl+O) → Entrée → Ctrl+X, je redemarre ensuite Frigate pour appliquer les changements :
cd /home/total/frigate → docker compose restart frigate

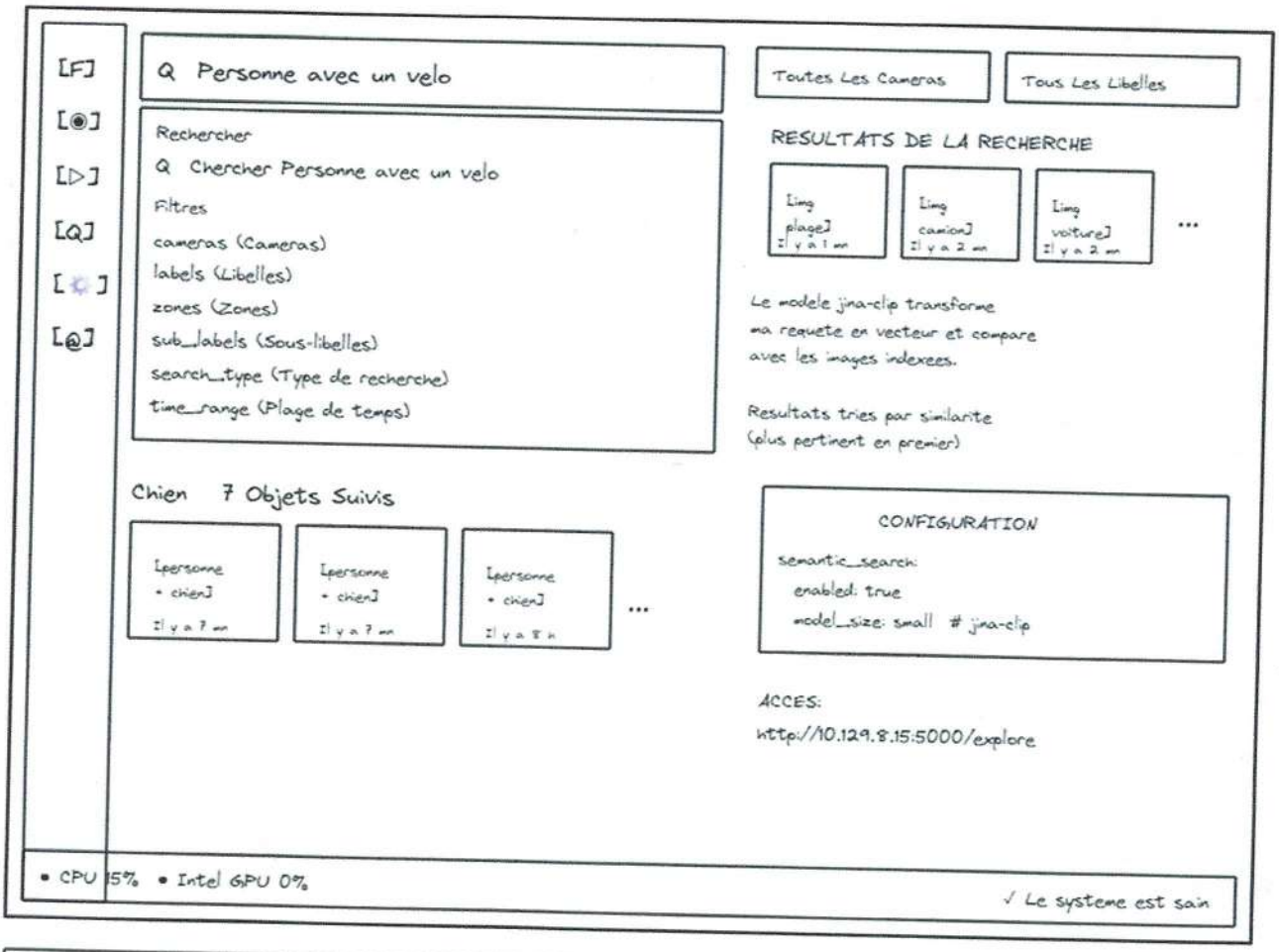
Comment fonctionne la recherche sémantique : le modèle jinav1 transforme chaque image détectée en un vecteur de nombres. Ce vecteur représente le contenu de l'image. Quand je tape une recherche comme "personne avec pantalon noir" ou filtre avec un label "label : keyboard", le système transforme aussi ma requête en vecteur. Il compare ensuite les deux vecteurs.. les images dont le vecteur est proche de ma requête apparaissent en premier dans les résultats.

Pour utiliser la recherche sémantique, j'ouvre l'int. web de Frigate : http://10.129.8.15:5000/explore/ ou je clique sur l'icone de loupe dans le menu de gauche.

En haut de la page, je vois une barre de recherche. Je tape ma requête en langage naturel. Par exemple : "Personne avec un vélo".

J'observe, quand je tape dans la barre de recherche, un menu déroulant apparaît. Ce menu propose l'option "chercher ---". En dessous, je vois la liste des filtres disponibles : caméras, labels, ---. Ces filtres permettent d'affiner la recherche.

CROQUIS DE L'INTERFACE RECHERCHE SEMANTIQUE



EXPLICATIONS

1. BARRE DE RECHERCHE : Je tape ma requete en langage naturel "Personne avec un velo"
2. MENU DEROULANT : Affiche les filtres disponibles (cameras, labels, zones, plage de temps...)
3. BOUTONS FILTRES : "Toutes Les Cameras", "Tous Les Libelles" permettent de filtrer les resultats
4. RESULTATS : Vignettes triees par pertinence avec horodatage (Il y a X mn/h)
5. CATEGORIES : Les objets sont regroupes par type (ex: "Chien 7 Objets Suivis")
6. BARRE D'ETAT : Affiche CPU 15%, Intel GPU 0%, "Le systeme est sain"

Les résultats s'affichent sous forme de vignettes. Chaque vignette montre une image avec son horodatage. Les images sont triées par pertinence. Les plus proches de ma recherche apparaissent en premier.

J'ai remarqué aussi que les résultats sont regroupés par catégorie. Par exemple, "Personne" 7 objets suivis" avec toutes les vignettes de personnes détectées. Chaque vignette montre les personnes détectées et l'horodatage de la détection.

En haut de l'interface, les boutons "Toutes les caméras" et "Tous les libellés" permettent de filtrer par caméra ou par type d'objet. En bas de l'écran, la barre d'état affiche "CPU 15%", "Le système est sain".

5.5. IA Générative de description sémantique

L'IA générative permet de créer automatiquement des descriptions textuelles des objets détectés. Par exemple, quand Frigate détecte une personne, l'IA peut décrire ses vêtements et ce qu'elle fait. Ces descriptions améliorent la recherche sémantique car je peux chercher des détails précis comme "personne en t-shirt rouge".

→ J'ai utilisé le service Google Gemini pour générer ces descriptions. Avant de configurer Frigate, j'ai obtenu une clé API gratuite sur Google AI Studio : → https://aistudio.google.com/ → je me connecte avec mon compte google → "Get API Key" dans le menu → je crée et je copie la clé API.

→ J'ouvre le fichier de configuration de Frigate : nano config.yml

→ J'ajoute la section genai :

```
genai:                                  → Ce paramètre active l'IA générative.
  enabled: true                         → Celui-ci indique que j'utilise le service google Gemini
  provider: gemini
  api_key: "AIzaSyBBJe09uYldZa9qrHwlZrsc2sWPbvX4-oA"   → Celui contient ma clé API.
  model: gemini-2.5-flash               → Spécifie le modèle à utiliser. rapide et gratuit, limité.
  prompt: "Analysez l'objet {label} dans ces images provenant de la caméra de sécurité {camera}. Concentrez-vous
sur les vetements. Répondez en français."
  object_prompts:
    person: "Examinez la personne principale dans ces images et ses vetements. Que fait-elle (par exemple,
s'approcher d'une porte, quitter une zone, rester immobile) ? Répondez en français."
```

Le paramètre prompt définit le texte envoyé à l'IA pour analyser les objets. Les variables {label} et {camera} sont remplacées automatiquement par le type d'objet et le nom de la caméra.

Le paramètre object_prompts permet de personnaliser le prompt pour chaque type d'objet. Ici, j'ai créé un prompt spécial pour les personnes qui demande de décrire les vêtements et les actions.

→ J'active sur la caméra NetCam2 :

```
cameras:
  NetCam2:                              → La section genai au niveau de la caméra active
    genai:                                la génération de descriptions pour cette caméra.
      enabled: true
      objects:                          → liste les types d'objet pour lesquels générer des descriptions.
        - person
      required_zones: []                → veut dire que toutes les zones de l'image sont concernées.
      min_score: 0.8                    → demande un score de confiance minimum de 80% avant de générer.
```

→ je sauvegarde et je redemarre : docker compose restart frigate.

Comment fonctionne l'IA générative : Quand Frigate détecte une personne sur NetCam2 avec un score > à 80%, il capture une image. Cette image est envoyée à l'API Google Gemini avec le prompt personnalisé. Gemini analyse l'image et génère une description en français. Cette description est stockée avec la détection. Elle apparaît dans le champ "Description de l'objet suivi" quand je clique sur une détection.

INTERFACE DETAILS OBJET AVEC GENAI

Details de l'objet suivi ×

Details | Instantane

Libelle
Personne

Meilleur score
94%

Score de l'instantane
92%

Camera
NetCam2

Horodatage
16 dec. 2025, 17:30 PM

[Image personne detectee par camera]

Description de l'objet suivi

Une personne portant un t-shirt bleu et un jean fonce marche vers la droite. Elle semble quitter la zone de la camera. La personne est de taille moyenne et ne porte pas de sac.

Description generee par Google Gemini (gemini-2.5-flash)

Generer a nouveau | Enregistrer

EXPLICATIONS

1. ONGLETS : "Details" affiche les infos, "Instantane" montre l'image complete
2. INFORMATIONS : Libelle (type objet), scores de confiance, camera source, horodatage
3. IMAGE : Apercu de la detection avec la personne detectee
4. DESCRIPTION GENAI : Texte genere par Google Gemini decrivant la personne, ses vetements et ses actions (en francais grace au prompt personnalise)
5. BOUTON "Generer a nouveau" : Demande une nouvelle description a l'IA
6. BOUTON "Enregistrer" : Sauvegarde les modifications

J'observe, quand je clique sur une détection de personne, une fenêtre "Détails de l'objet suivi" s'ouvre. En bas de la fenêtre, je vois le champ "Description de l'objet suivi". Ce champ contient la description générée par Gemini. Par exemple. Voir le croquis.

5.6. Autre modèle de réseaux de neurones entraîné

L'objectif de cette partie était d'entrainer un modèle de détection personnalisé pour reconnaître l'objet "bombe" et l'intégrer dans Frigate. J'ai effectué plusieurs tentatives avec différentes approches.

J'ai préparé un dataset d'images de bombes sur la plateforme Robotflow. J'ai annoté les images en dessinant des rectangles autour de chaque objet. J'ai entraîné plusieurs modèles avec différentes versions de YOLO: YOLOv5, YOLOv8 et YOLOv9. Les résultats d'entraînement étaient excellents. J'ai exporté les modèles au format ONNX pour les utiliser dans Frigate. J'ai même crée un script Python de détection externe pour contourner les limitations.

Malgré tous ces efforts, l'intégration dans Frigate n'a pas fonctionné. Le modèle entraîné était bon, mais Frigate ne pouvait pas l'utiliser pour détecter ma classe personnalisée "bombe".

→ Limitation technique identifiée:

Après de recherches dans la documentation officielle de Frigate et sur les forums de la communauté, j'ai identifié la cause de l'échec. La documentation de Frigate indique clairement:

"You cannot add new object types, but you can change the names of existing objects in the model.", ...

Ça explique tout. Frigate est conçu pour fonctionner uniquement avec les classes du dataset COCO. Le dataset COCO contient 80 classes d'objets comme "person", "car", "dog", "cat", "mouse", etc. La classe "bombe" n'existe pas dans COCO.

Frigate ne permet pas d'ajouter une classe personnalisée comme "bombe", "arme" ou tout autre objet absent du dataset COCO. Même si j'entraîne un modèle parfait avec 99,5% de précision, Frigate ne sait pas comment traiter cette nouvelle classe.

Par contre, le paramètre "labelmap" dans la configuration permet de renommer une classe existante. Par exemple, je peux renommer "truck"

en "vehicule" ou "cat" en "animal". Mais je ne peux pas créer une nouvelle classe qui n'existe pas dans le modèle de base.

→ Pourquoi cette limitation ?

Frigate est optimisé pour la vidéosurveillance résidentielle et professionnelle standard. Les développeurs ont choisi de supporter les 80 classes CoCo car elles couvrent les besoins courants : personnes, véhicules, animaux, objets du quotidien. Ajouter le support des classes personnalisées complexifierait beaucoup le système.

En somme, Cette partie du projet n'a pas abouti à cause de cette limitation technique. Cette expérience m'a appris l'importance de vérifier les limitations techniques d'un logiciel avant de commencer un développement. L'entraînement du modèle était un succès, mais l'intégration était impossible avec l'architecture actuelle de Frigate.

Cette limitation est documentée sur le site officiel de Frigate. La page "Advanced Options" précise que le label map sert uniquement à renommer des classes existantes, pas à en créer de nouvelles. Pour intégrer des objets personnalisés, il faudrait soit modifier le code source de Frigate, soit utiliser un autre système de vidéosurveillance qui supporte les modèles personnalisés, soit attendre une future version de Frigate avec cette fonctionnalité.

Intégration MQTT pour recevoir les notifications de détection.

En plus des fonctionnalités demandées, j'ai mis en place une intégration MQTT. MQTT est un protocole de messagerie léger. Il permet de recevoir des notifications en temps réel quand Frigate détecte un objet ou reconnait un visage. Cette fonctionnalité est utile pour créer des alertes ou connecter Frigate à d'autres systèmes comme Home Assistant.

Mosquitto est un broker MQTT. C'est le serveur qui reçoit et distribue les messages. J'installe Mosquitto et ses outils clients sur le PC serveur (conteneur Frigate et pc hôte) :

```
apt install mosquitto mosquitto-clients -y
```

installe le serveur (broker) MQTT. le paquet "mosquitto-clients" installe les outils en ligne de commande pour publier et recevoir des messages.

→ Je démarre le service et j'active au démarrage :

```
systemctl start mosquitto
systemctl enable mosquitto
systemctl status mosquitto
```

→ J'ouvre le fichier de configuration de Frigate : nano config.yml. J'ajoute la section mqtt au début du fichier :

```
mqtt:
  enabled: true
  host: 172.17.0.1
  port: 1883
  topic_prefix: frigate
  client_id: frigate
  stats_interval: 60
```

- enabled: true → active l'envoi des messages MQTT. Le paramètre "host" est l'adresse IP du bridge Docker.
- host: 172.17.0.1 → Cette adresse permet au conteneur Frigate d'accéder au broker mosquitto qui tourne sur le PC hôte.
- port: 1883 → port standard.
- topic_prefix: frigate → définit le préfixe des topics. Tous les messages commenceront par "frigate/".
- client_id: frigate → Ce paramètre est l'identifiant du client MQTT.
- stats_interval: 60 → Envoie des statistiques toutes les 60 sec.

→ Par défaut, Mosquitto n'écoute que sur localhost. Je modifie sa configuration pour accepter les connexions depuis le conteneur Docker : nano /etc/mosquitto/mosquitto.conf, j'ajoute :

```
listener 1883 0.0.0.0
allow_anonymous true
```

- listener 1883 0.0.0.0 → fait écouter Mosquitto sur toutes les interfaces réseau.
- allow_anonymous true → permet les connexions sans mot de passe.

→ Je redémarre Mosquitto puis Frigate : Systemctl restart mosquitto, Cd /home/tatol/Frigate → docker compose restart frigate.

→ J'ouvre un terminal et je m'abonne à tous les messages Frigate :

```
mosquitto_sub -h 127.0.0.1 -t "frigate/#" -v
```

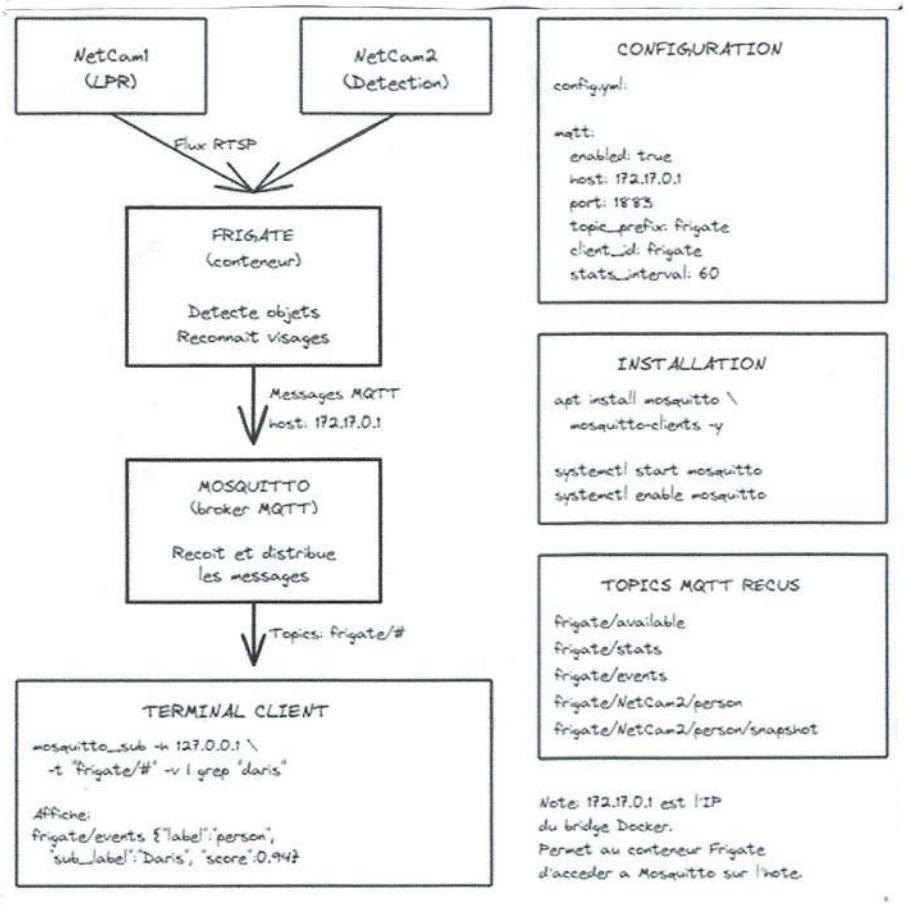
Cette commande permet de s'abonner aux messages. L'option -h 127.0.0.1 indique l'adresse du broker. L'option -t "frigate/#" signifie "tous les topics qui commencent par frigate/". Le dièse (#) est un joker qui capture tous les sous-topics. L'option -v affiche le nom du topic avec le message.

→ Pour ne voir que les messages concernant un personne reconnue, par exemple moi : Daris, j'utilise grep :

mosquitto_sub -h 127.0.0.1 -t "frigate/#" -v | grep -i "daris", la commande "grep -i "daris" " filtre les messages pour n'afficher que ceux contenant le nom "Daris". L'option `-i` ignore la casse (majuscules/minuscules).

Types de messages reçus par MQTT :

- frigate/available : "Online" : quand Frigate démarre
- frigate/stats : statistiques système (CPU, mémoire)
- frigate/events : Tous les événements de détection
- frigate/NetCam2/person : Compteur de personnes sur NetCam2
- frigate/NetCam2/person/snapshot : Image de la détection en base64



J'observe, quand je passe devant la NetCam 2, j'vois immédiatement des messages apparaît dans le terminal.

Quand je filtre avec grep sur "Daris", je ne vois que les détections où mon visage a été reconnu.

Cela permet de créer facilement des alertes personnalisées. Par exemple, je pourrais envoyer une notification sur mon téléphone uniquement quand une personne spécifique est détectée.

Conclusion

Ce projet SAE 5.ROM.03 m'a permis de déployer un système complet de vidéo surveillance intelligent avec Frigate.VMS. J'ai installé le serveur en utilisant Docker pour conteneuriser l'application. J'ai configuré deux caméras IP Hikvision qui envoient leurs flux vidéo en RTSP vers le serveur. Le système détecte les personnes et les claviers grâce à deux détecteurs CPU qui analysent les images en temps réel.

J'ai mis en place plusieurs fonctionnalités avancées avec succès. La lecture de plaques d'immatriculation fonctionne très bien. J'ai testé avec la plaque RT-1994-LP et obtenu un score de confiance élevé. La reconnaissance faciale avec l'algorithme FaceNet permet d'identifier les visages enregistrés dans la bibliothèque. J'ai pu ajouter des visages directement depuis l'interface web et le système les reconnaît avec des scores supérieurs à 90%. La recherche sémantique avec le modèle jinav1 permet de retrouver des événements passés en tapant des requêtes simples comme "personne avec un velo". L'IA générative Google Gemini génère automatiquement des descriptions en français des personnes détectées. Ces descriptions enrichissent la base de données et améliorent la recherche. J'ai aussi intégré le protocole MQTT avec le broker Mosquitto. Je reçois maintenant des notifications en temps réel dans un terminal quand une personne est détectée. Je peux filtrer ces messages avec grep pour ne voir que les alertes concernant une personne précise.

Une seule partie du projet n'a pas abouti. J'ai entraîné des modèles YOLOv5, YOLOv8 et YOLOv9 pour détecter des bombes. Les résultats d'entraînement étaient excellents avec 99.5% de précision. Mais l'intégration dans Frigate a échoué. La documentation officielle explique que Frigate ne supporte pas l'ajout de classes d'objets personnalisées en dehors du dataset COCO. Cette limitation technique m'a appris à toujours vérifier la compatibilité d'un logiciel avant de commencer un développement.

Sur le plan personnel, ce projet a été très stimulant. La partie que j'ai le plus aimée, c'est la recherche. Aller consulter la documentation officielle de Frigate. Chercher des solutions sur les forums et les discussions en ligne. Utiliser l'intelligence artificielle pour comprendre des parties et concepts complexes. Chaque fois que je bloquais sur un problème, je devais creuser pour trouver la réponse. Et quand la solution marchait enfin, je ressentais une vraie satisfaction.

J'ai beaucoup appris pendant ce projet. J'ai appris la rigueur. Un fichier de configuration avec une erreur de syntaxe, et le conteneur Docker refuse de démarrer. Une mauvaise adresse IP, et la caméra ne se connecte pas. Cette rigueur m'a forcé à lire attentivement chaque ligne de code et à comprendre ce que je faisais.. J'ai aussi développé des méthodes de travail. J'utilise Notion pour organiser mes projets, noter mes avancées et garder une trace de mes apprentissages. Ces méthodes me serviront dans ma future carrière.

Ce que je retiens surtout, c'est le sentiment de progression. Au début du projet, je ne connaissais pas Frigate. A la fin, je maîtrise la configuration des caméras, la détection d'objets, la reconnaissance faciale, la recherche sémantique, l'IA générative et l'intégration MQTT. Chaque nouvelle fonctionnalité qui marchait me donner envie d'aller plus loin. Ce projet m'a donné confiance en ma capacité à apprendre des technologies nouvelles et à résoudre des problèmes techniques complexes.